

Pedro Coutinho da Silva

**Relatório Técnico Expandido (Trilha B): Análise
Experimental e Estatística da Hierarquia de Memória,
Subsistemas de I/O e Paralelismo em Processador
Híbrido Intel Core i5-13420H**

CESAR School

Curso de Engenharia de Computação

Disciplina de Infraestrutura de Hardware

Orientador: Prof. Ronierison Maciel

Recife, PE

2026

Sumário

1	SUMÁRIO EXECUTIVO	3
2	INTRODUÇÃO	4
3	FUNDAMENTAÇÃO TEÓRICA E TRABALHOS RELACIONADOS	5
3.1	Contexto Bibliográfico	5
3.2	Arquitetura Híbrida Intel Raptor Lake	5
3.3	Hierarquia de Memória, Paginação e TLB	6
3.4	Interrupções Virtualizadas no WSL2	6
4	METODOLOGIA	7
4.1	Ambiente Experimental	7
4.2	Protocolo Estatístico	7
4.3	Scripts de Teste	8
5	RESULTADOS E ANÁLISE	9
5.1	Escalabilidade de Paralelismo	9
5.2	Localidade Espacial de Referência	9
5.3	Workload Integrado (CPU, Memória e I/O)	10
6	DISCUSSÃO, AMEAÇAS À VALIDADE E RECOMENDAÇÕES .	11
6.1	Análise Crítica dos Gargalos	11
6.2	Ameaças à Validade	11
6.3	Recomendações Técnicas	11
7	CONSIDERAÇÕES FINAIS	13
	REFERÊNCIAS	14
	ANEXOS	16
	ANEXO A – ARTEFATOS DO REPOSITÓRIO	17

1 Sumário Executivo

Este relatório apresenta uma investigação técnica empírica e estatisticamente fundamentada sobre o comportamento microarquitetural do processador heterogêneo Intel Core i5-13420H (Raptor Lake, 13^a geração) em um ambiente Ubuntu 24.04.3 LTS virtualizado via WSL2 sobre Windows 11/Hyper-V. A motivação central reside no mapeamento das perdas de desempenho que afetam tarefas de engenharia de software em ambientes virtuais de desenvolvimento pessoal com arquiteturas híbridas P-core/E-core.

Os resultados obtidos em 30 amostras válidas (após 5 repetições de *warm-up*) apoiam-se em três eixos analíticos:

1. **Saturação de Paralelismo:** Escalar de 1 para 4 threads reduziu o tempo de execução de 0,9804 s para 0,6242 s ($-36,33\%$). Ao saturar com 12 threads, o tempo estabilizou em 0,6414 s, sem melhoria significativa ($p\text{-valor} = 0,0663 > 0,05$), evidenciando a barreira do barramento DDR4 em provável configuração de canal simples.
2. **Localidade Espacial:** O percurso por colunas em uma matriz de 128 MB gerou overhead de 4,11% sobre o percurso por linhas ($p\text{-valor} = 0,0281 < 0,05$), confirmando penalidade sistemática associada à degradação de *prefetch* e de localidade de *cache*/TLB.
3. **Overhead de Virtualização:** O subsistema de I/O em disco virtual (.vhdx) registrou desvio-padrão de 91,23 MB/s nas taxas de transferência, evidenciando flutuações crônicas decorrentes de interrupções paravirtualizadas via VMBus/Hyper-V. Adicionalmente, o SSD NVMe opera em PCIe Gen 3x4 em vez do Gen 4x4 suportado pelo hardware.

Recomenda-se limitar aplicações matriciais a no máximo 4 threads e evitar o WSL2 para cargas de alta transatividade de I/O.

2 Introdução

A evolução da engenharia de computadores conciliou potência aritmética com limitações de dissipação térmica. Após o esgotamento do escalonamento de Dennard, a transição para sistemas *multicore* consolidou-se, mas encontrou a barreira da memória (*memory wall*) (MCCALPIN, 1995) — conceitos fundamentados em Patterson e Hennessy (2021) e Hennessy e Patterson (2017). Recentemente, a Intel adotou topologias híbridas, integrando P-cores de alto desempenho (Golden Cove, com Hyper-Threading) e E-cores de eficiência energética (Gracemont) no mesmo *die*. O i5-13420H materializa esse paradigma com 4 P-cores (8 threads lógicas) e 4 E-cores (4 threads lógicas), totalizando 12 unidades lógicas de processamento.

No contexto de desenvolvimento moderno, ambientes como WSL2 ocultam um custo computacional silencioso: o hipervisor Hyper-V insere indireções no acesso ao hardware físico, alterando a dinâmica de caches, paginação e tratamento de interrupções. Nesse cenário, a aceleração paralela linear prevista pela Lei de Amdahl (AMDAHL, 1967) afasta-se da realidade observada.

Este relatório testa formalmente as seguintes hipóteses:

- H_0 : O tempo de execução reduz-se linearmente com o número de threads ($T(N) \propto T(1)/N$), sem limitação por saturação do barramento DRAM.
- H_1 : A largura de banda DDR4 em canal simples atua como gargalo compartilhado insuperável. Acima de 4 threads (P-cores físicos), a alocação de threads adicionais não produz ganho temporal estatisticamente significativo.

A validação baseia-se em 30 amostras válidas, intervalos de confiança (IC) a 95% e testes t de Student bicaudais ($\alpha = 0,05$).

3 Fundamentação Teórica e Trabalhos Relacionados

3.1 Contexto Bibliográfico

A disparidade entre velocidade da CPU e taxa de transferência da DRAM foi descrita seminalmente por [McCalpin \(1995\)](#) via suíte STREAM, demonstrando saturação assintótica da largura de banda após poucas unidades paralelas. [McVoy e Staelin \(1996\)](#) propuseram o LMBench para mapear latências ao longo da hierarquia de memória. O modelo Roofline de [Williams, Waterman e Patterson \(2009\)](#) classifica aplicações como *compute-bound* ou *memory-bound* com base na intensidade aritmética (FLOP/byte).

A literatura de arquiteturas *multicore* estabelece que a largura de banda de memória compartilhada se torna o fator limitante do escalonamento à medida que o número de núcleos ativos cresce. [Rogers et al. \(2009\)](#) formalizaram o problema da *bandwidth wall*, mostrando que a banda disponível por núcleo decresce conforme se multiplicam as unidades de processamento em um mesmo *die*. [Diamond et al. \(2011\)](#) demonstraram experimentalmente que, uma vez saturado o subsistema de memória, o padrão e a ordem dos acessos passam a determinar o desempenho, e que métricas tradicionais de núcleo único podem ser enganosas no regime *multicore*. No domínio das arquiteturas híbridas, a documentação técnica da Intel ([Intel Corporation, 2021](#)) descreve a coexistência de P-cores e E-cores e os mecanismos de escalonamento associados, contexto em que a contenção pelo canal de memória compartilhado tende a anular ganhos esperados do paralelismo em *workloads* intensivos em memória. A teoria de paginação e alocação tardia é fundamentada em [Tanenbaum e Bos \(2015\)](#) e [Love \(2010\)](#).

3.2 Arquitetura Híbrida Intel Raptor Lake

A microarquitetura Raptor Lake ([Intel Corporation, 2021](#); [Intel Corporation, 2022a](#)) integra dois tipos de núcleos:

P-cores (Golden Cove): Decodificador de 6 instruções/ciclo, ROB de 512 entradas, 12 portas de execução, Hyper-Threading (SMT 2-vias) e pipeline de 13 estágios. Otimizados para minimizar latência instrução por instrução.

E-cores (Gracemont): ROB de 256 entradas, sem Hyper-Threading, menor suporte vetorial. Grupos de 4 E-cores compartilham 2 MiB de cache L2. Maximizam rendimento por watt e por área de silício.

O *Intel Thread Director* ([Intel Corporation, 2022b](#)) monitora classes de instruções por thread e sinaliza ao escalonador do SO a migração de tarefas vetoriais intensas para P-

cores. No WSL2, a VM Linux depende de sinais retransmitidos pelo Hyper-V, introduzindo atraso residual nessas migrações.

3.3 Hierarquia de Memória, Paginação e TLB

No i5-13420H, cada P-core possui L1d de 48 KiB e L2 privado de 2 MiB; os E-cores compartilham 2 MiB de L2 por *cluster*; o LLC (L3) é de 12 MiB compartilhado. O controlador de memória integrado (IMC), em provável configuração de canal simples DDR4 estimada em 3200 MT/s conforme a especificação da plataforma, limita a taxa teórica agregada a $\approx 25,6$ GB/s, compartilhada entre todos os 12 threads.

O kernel do Linux utiliza paginação de quatro níveis (PML4 $\rightarrow$ PDPT $\rightarrow$ PD $\rightarrow$ PT) com alocação tardia (*lazy allocation*): frames físicos são alocados apenas na ocorrência de *page fault*. O TLB armazena traduções recentes em silício; percursos por colunas com saltos maiores que 32 KiB tendem a reduzir o reaproveitamento de traduções e a eficácia do *prefetcher*, adicionando latência mensurável.

3.4 Interrupções Virtualizadas no WSL2

Em hardware nativo, dispositivos emitem sinais MSI/MSI-X diretamente ao APIC. No WSL2, toda comunicação de I/O é mediada pelo VMBus: a requisição gera um *VM-exit*, o Hyper-V processa a operação no arquivo `.vhdx` físico, recebe a interrupção real do SSD e retorna via *hypervisor callback* (*VM-enter*). Essa cadeia insere latência e flutuação crônicas no subsistema de disco virtual.

4 Metodologia

4.1 Ambiente Experimental

Hardware (host físico):

- Processador: Intel Core i5-13420H — 4 P-cores (8 threads, turbo 4,6 GHz) + 4 E-cores (4 threads, turbo 3,4 GHz) = 12 threads lógicas.
- Cache: L1i 288 KiB | L1d 192 KiB | L2 7,5 MiB | L3 12 MiB (LLC compartilhada).
- RAM: 8 GiB DDR4 instalados (7,6 GiB úteis ao sistema), em provável configuração de canal simples estimada em 3200 MT/s conforme especificação da plataforma.
- SSD NVMe: SK Hynix (PCI 1C5C:1D59), link negociado em PCIe Gen 3x4 ([BUDRUK](#); [ANDERSON](#); [SHANLEY, 2003](#)).
- GPU: Nvidia RTX 3050 Laptop (PCI 10DE:28A1), link PCIe Gen 4x8.

Software (ambiente guest):

- Host: Windows 11 Home (64 bits) com Hyper-V.
- Guest: Ubuntu 24.04.3 LTS via WSL2, kernel 6.6.87.2-microsoft-standard-WSL2.
- Linguagem: Python 3.12.3 com NumPy 1.26.4 e SciPy 1.11.4.

4.2 Protocolo Estatístico

Para isolar o comportamento do hardware frente ao ruído do SO hospedeiro, adotou-se:

1. **Warm-up**: 35 repetições por teste; as 5 primeiras descartadas para estabilizar caches e forçar saída de C-states profundos.
2. **Amostra**: $N = 30$ repetições válidas e consecutivas.
3. **IC a 95%**: $IC = \bar{X} \pm t_{0,025,29} \cdot (SD/\sqrt{n})$, com $t_{0,025,29} \approx 2,0452$.
4. **Teste de hipótese**: Teste t de Student bicaudal para duas amostras independentes; limiar $\alpha = 0,05$.

4.3 Scripts de Teste

- `teste_paralelismo.py`: Paraleliza 12 tarefas matriciais (`np.random.rand(1250, 1000)`) variando o pool em $N \in \{1, 2, 4, 12\}$ processos. Compara estatisticamente $N = 4$ vs. $N = 12$.
- `teste_localidade.py`: Percorre uma matriz de 128 MB (4000×4000 *double floats*) por linhas (contíguo) e por colunas (saltos). Aplica teste t entre os dois modos.
- `workload_completo.py`: Executa sequencialmente uma fase CPU-bound (cálculo vetorial intenso) e uma fase Memory-bound (cópias de blocos acima do LLC de 12 MiB), seguida de fase I/O-bound em disco virtual.

5 Resultados e Análise

5.1 Escalabilidade de Paralelismo

A Tabela 1 apresenta os tempos de execução medidos para as quatro configurações de paralelismo.

Tabela 1 – Tempos de execução sob escala de paralelismo ($N = 30$)

Threads/Processos	Média (s)	Desvio-Padrão (SD)	IC 95% Inferior (s)	IC 95% Superior (s)
1	0,9804	0,0837	0,9492	1,0117
2	0,7180	0,0462	0,7007	0,7353
4	0,6242	0,0313	0,6125	0,6359
12	0,6414	0,0393	0,6267	0,6561

O decréscimo de tempo é consistente de 1 para 4 threads: a redução de 36,33% confirma o ganho real ao saturar os P-cores físicos. Porém, ao expandir para 12 processos, o tempo médio subiu marginalmente para 0,6414 s (+2,76%) e o desvio-padrão aumentou de 0,0313 para 0,0393, indicando maior instabilidade. O teste t bicaudal entre $N = 4$ e $N = 12$ gerou p-valor = 0,0663. Como $0,0663 > \alpha = 0,05$, **falha-se em rejeitar** H_0 de igualdade de médias: a diferença não é estatisticamente significativa. Isso comprova que o barramento DDR4 em canal simples atinge saturação com 4 threads, tornando a alocação de threads adicionais ineficaz.

5.2 Localidade Espacial de Referência

A Tabela 2 reúne os dados do teste de percurso matricial.

Tabela 2 – Tempos de acesso no teste de localidade de cache ($N = 30$)

Forma de Acesso	Média (s)	Desvio-Padrão (SD)	IC 95% Inferior (s)	IC 95% Superior (s)
Linhas (Sequencial)	2,9871	0,2391	2,8978	3,0764
Colunas (Saltos)	3,1098	0,1787	3,0431	3,1766

O acesso por colunas superou em 4,11% o percurso por linhas. O teste t bicaudal revelou p-valor = $0,0281 < 0,05$, confirmando com significância formal a existência de uma penalidade sistemática associada ao padrão de acesso não-contíguo. Cabe ressaltar que, por percorrer a matriz elemento a elemento em laços Python, o custo dominante de ambos os percursos é o *overhead* do interpretador; ainda assim, o efeito de localidade sobrevive estatisticamente a esse ruído. A Tabela 3 detalha a distribuição amostral desse efeito.

Enquanto 83,3% das rodadas por linhas ficaram abaixo de 3,10 s, no modo colunas essa proporção cai para 46,6%, evidenciando o deslocamento sistemático da distribuição para faixas de latência maiores.

Tabela 3 – Distribuição de frequência dos tempos no teste de localidade

Intervalo (s)	Freq. Linhas	Freq. Colunas
$2,50 \leq T < 2,80$	5	0
$2,80 \leq T < 3,10$	20	14
$3,10 \leq T < 3,40$	5	13
$3,40 \leq T < 3,70$	0	3
Total	30	30

5.3 Workload Integrado (CPU, Memória e I/O)

A Tabela 4 sintetiza os resultados das três fases do script integrado.

Tabela 4 – Resultados estatísticos das fases do workload integrado ($N = 30$)

Fase	Média	Desvio-Padrão	IC 95% Inf.	IC 95% Sup.
CPU-Bound (Mop/s)	3.792,03	453,89	3.622,54	3.961,52
Memory-Bound (GB/s)	7,07	1,35	6,56	7,57
I/O-Bound (MB/s)	649,93	91,23	615,87	684,00

Na fase CPU-bound, a taxa de 3.792,03 Mop/s com desvio de 453,89 Mop/s reflete o escalonamento dinâmico em ambiente virtualizado. A fase Memory-bound atingiu 7,07 GB/s; esse valor corresponde à banda sustentada por uma única *stream* de cópia sequencial (a operação `np.copyto` executa em uma única thread), e não à saturação agregada do canal. O teto teórico agregado do canal simples DDR4 ($\approx 25,6$ GB/s) só seria aproximado com múltiplos fluxos concorrentes; a banda de 7,07 GB/s de um único fluxo é, portanto, coerente com o regime *memory-bound* observado. A fase I/O-bound apresentou o maior coeficiente de variação (14%), com desvio de 91,23 MB/s, diretamente associado à cadeia *VM-exit*/Hyper-V/*VM-enter* nas interrupções virtualizadas do subsistema de disco `.vhd\`.

6 Discussão, Ameaças à Validade e Recomendações

6.1 Análise Crítica dos Gargalos

Os ensaios comprovam que o i5-13420H, neste ambiente, é predominantemente *memory-bound*. Com 4 processos matriciais paralelos, o IMC atinge o limite dinâmico de vazão DDR4. Alocar 8 threads adicionais mantém as unidades aritméticas em estado ocioso (*stalled*), aguardando resolução de leituras na DRAM física — fenômeno previsto pelo modelo Roofline (WILLIAMS; WATERMAN; PATTERSON, 2009) e consistente com a contenção de banda compartilhada descrita em Rogers et al. (2009) e Diamond et al. (2011). O overhead de 4,11% no percurso por colunas decorre da degradação das predições do *prefetcher* e da menor localidade de *cache*/TLB no acesso não-contíguo; *page table walks* adicionais na DRAM constituem um dos fatores plausíveis, ainda que o efeito esteja parcialmente atenuado pelo pré-buscador do i5-13420H e mascarado pelo *overhead* do interpretador. A assinatura temporal, contudo, permanece estatisticamente mensurável.

6.2 Ameaças à Validade

1. **Carga de plano de fundo do Windows 11:** Antivírus em tempo real, atualizações e renderização da GPU concorrem pelo barramento DRAM, ampliando o desvio-padrão amostral.
2. **Overhead WSL2/Hyper-V:** A emulação de interrupções via VMBus insere variações crônicas, justificando o desvio de 91,23 MB/s na fase I/O.
3. **Escalonamento híbrido e térmico:** Sem acesso direto ao *Intel Thread Director*, o escalonador Linux convidado pode desviar tarefas vetoriais para E-cores. Adicionalmente, o *Thermal Throttling* (limites PL1 de 45 W e PL2 de 95 W por ≈ 28 s) reduz a frequência de clock durante baterias longas, aumentando a dispersão amostral. O protocolo de *warm-up* mitigou os efeitos de C-states, mas não elimina a variabilidade térmica.

6.3 Recomendações Técnicas

1. **Limitar pools matriciais a 4 threads:** O canal DDR4 simples satura com 4 processos. Threads adicionais geram ociosidade forçada de CPU sem ganho mensurável.
2. **Evitar WSL2 para I/O de alta transatividade:** Bancos de dados e workloads

intensivos em disco devem rodar em instalação nativa (Windows NT ou Linux *bare metal*), eliminando a cadeia *VM-exit/VM-enter*.

3. **Fixar afinidade nos P-cores:** Verificar o mapeamento real dos núcleos com `lscpu` -e antes de usar `sched_setaffinity`; em seguida, travar threads críticas nos índices lógicos correspondentes aos P-cores, impedindo desvio para os E-cores de menor desempenho vetorial. Note-se que, sob WSL2, a numeração lógica pode não seguir a ordem física esperada.
4. **Garantir acesso sequencial por linhas:** Em loops NumPy, a travessia orientada a linhas (padrão C contíguo) minimiza *cache misses* e *TLB misses*, evitando a penalidade estatisticamente comprovada de 4,11%.
5. **Auditar o link PCIe do SSD NVMe:** Verificar na BIOS se o link está negociado em Gen 4x4 (até ≈ 8 GB/s) em vez de Gen 3x4 (cujo limite prático é de $\approx 3,5$ GB/s úteis, ou $\approx 3,94$ GB/s brutos), desativando estados agressivos de economia de energia de barramento (PCIe L1) se necessário. Observa-se que o *throughput* sequencial medido pelo fio (1.527 MB/s) está muito abaixo do limite do link Gen 3x4, o que indica que o gargalo do I/O sequencial é a camada de virtualização do WSL2, e não a geração do enlace PCIe.

7 Considerações Finais

Este relatório técnico realizou a caracterização física e de software da plataforma Intel Core i5-13420H sob virtualização WSL2/Hyper-V. Com protocolo de 30 amostras válidas, ICs a 95% e testes t de Student bicaudais ($\alpha = 0,05$), foram avaliados: escalabilidade de paralelismo, penalidades de localidade de cache/TLB e flutuações de I/O virtualizado.

Os testes de paralelismo rejeitam a hipótese de aceleração linear (H_0), corroborando H_1 : o barramento DDR4 de canal único limita a vazão por fluxo a $\approx 7,07$ GB/s, tornando ineficaz ($p = 0,0663$) a expansão para 12 threads. A análise de localidade confirma overhead sistemático de 4,11% no percurso por colunas ($p = 0,0281$), evidenciando o custo real do acesso não-contíguo à memória. O subsistema de I/O virtualizado exhibe flutuação expressiva (CV de 14%) pela cadeia de chaveamentos Hyper-V.

Como trabalho futuro, recomenda-se aplicar o mesmo protocolo em instalação *bare metal* do Linux, permitindo separar as penalidades exclusivas da virtualização das restrições inerentes à arquitetura heterogênea e ao controle térmico do i5-13420H.

Referências

AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. In: *Proceedings of the Spring Joint Computer Conference (AFIPS)*. New York, NY: ACM, 1967. p. 483–485.

BUDRUK, R.; ANDERSON, D.; SHANLEY, T. *PCI Express System Architecture*. Boston, MA: Addison-Wesley, 2003.

DIAMOND, J. et al. Evaluation and optimization of multicore performance bottlenecks in supercomputing applications. In: *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Austin, TX: IEEE, 2011. p. 32–43.

HENNESSY, J. L.; PATTERSON, D. A. *Computer Architecture: A Quantitative Approach*. 6. ed. Cambridge, MA: Morgan Kaufmann, 2017.

Intel Corporation. *Intel Performance Hybrid Architecture and Software Optimizations, Part One: Introduction*. [S.l.], 2021. Disponível em: <https://cdrdv2-public.intel.com/685861/211115_Hybrid_WP_1_Introduction_v1.2.pdf>.

Intel Corporation. *13th Generation Intel Core Processors: Datasheet, Volume 1*. [S.l.], 2022. Disponível em: <<https://www.intel.com/content/www/us/en/products/docs/processors/core/13th-gen-core-datasheet-vol-1.html>>.

Intel Corporation. *Intel Thread Director: Delivering Optimal Performance for Hybrid Architecture*. [S.l.], 2022. Disponível em: <<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-thread-director.html>>.

LOVE, R. *Linux Kernel Development*. 3. ed. Upper Saddle River, NJ: Addison-Wesley, 2010.

MCCALPIN, J. D. *STREAM: Sustainable Memory Bandwidth in High Performance Computers*. [S.l.], 1995. Disponível em: <<https://www.cs.virginia.edu/stream/>>.

MCVOY, L.; STAELIN, C. lmbench: Portable tools for performance analysis. In: *Proceedings of the USENIX Annual Technical Conference*. San Diego, CA: USENIX Association, 1996. p. 279–294.

PATTERSON, D. A.; HENNESSY, J. L. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. 2. ed. Cambridge, MA: Morgan Kaufmann, 2021.

ROGERS, B. M. et al. Scaling the bandwidth wall: Challenges in and avenues for CMP scaling. In: *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA)*. New York, NY: ACM, 2009. p. 371–382.

TANENBAUM, A. S.; BOS, H. *Organização Estruturada de Computadores*. 6. ed. São Paulo: Pearson, 2015.

WILLIAMS, S.; WATERMAN, A.; PATTERSON, D. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, ACM, v. 52, n. 4, p. 65–76, 2009.

Anexos

ANEXO A – Artefatos do Repositório

Para fins de reprodutibilidade, todos os dados brutos coletados e os scripts utilizados nos experimentos encontram-se disponíveis em repositório público. Os artefatos estão organizados nas pastas **dados/** (medições originais) e **scripts/** (implementações em Python com protocolo estatístico).

Arquivos de dados (dados/):

- `info_cpu.txt` — saída de `lscpu` (identificação da CPU);
- `saida_paralelismo.txt` — médias, IC 95% e p-valor do teste de paralelismo;
- `saida_localidade.txt` — IC 95% e p-valor do teste de localidade;
- `saida_workload.txt` — médias e IC 95% das fases do workload integrado;
- `medicoes_mbw.txt` — largura de banda de memória (`mbw`);
- `medicoes_fio.txt` — benchmark de armazenamento (`fio`);
- `medicoes_memoria.txt` — `free`, `vmstat` e `/proc/PID/status`;
- `medicoes_pcie.txt` — links PCIe (`LnkCap` e `LnkSta`);
- `medicoes_interrupcoes.txt` — contagem de interrupções durante I/O;
- `cinebench_r23.txt` — pontuações do Cinebench R23.

Scripts (scripts/):

- `teste_paralelismo.py`;
- `teste_localidade.py`;
- `workload_completo.py`.

Repositório: <<https://github.com/Pedro-Coutinho2612/infra-hw-artigo>>